

# Computer Vision Practical Report:

## GRADIENT TECHNIQUE

### 1. AIM

To compute the Morphological Gradient (the difference between dilation and erosion of an image) to highlight structural boundaries and object outlines using `cv2.morphologyEx()` in OpenCV, display the results, and save the output image.

### 2. PROCEDURE

- Load the input image using `cv2.imread()` and convert it to grayscale using `cv2.cvtColor()`.
- Define a spatial structuring element (kernel) using `cv2.getStructuringElement()` (e.g., rectangular or elliptical).
- Apply the morphological gradient operation using `cv2.morphologyEx()` with `cv2.MORPH_GRADIENT`.
- Display the original and morphological gradient images side-by-side using Matplotlib.
- Save the output image to disk using `cv2.imwrite()`.

### 3. PROGRAM CODE

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Load image
img_bgr = cv2.imread('E33-IMG.jpg')
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

# 2. Define Structuring Element (5x5 kernel)
kernel = np.ones((5, 5), np.uint8)

# 3. Perform Morphological Gradient (Dilation - Erosion)
gradient = cv2.morphologyEx(img_rgb, cv2.MORPH_GRADIENT, kernel)
```

# 4. Display side-by-side on a SINGLE PAGE

```
fig, axes = plt.subplots(1, 2, figsize=(8, 4))
```

```
axes[0].imshow(img_rgb)
```

```
axes[0].set_title('Original Image')
```

```
axes[0].axis('off')
```

```
axes[1].imshow(gradient)
```

```
axes[1].set_title('Morphological Gradient')
```

```
axes[1].axis('off')
```

```
plt.tight_layout()
```

```
plt.show()
```

## 4. OUTPUT

Original Image



Morphological Gradient



## 5. RESULT

The Python script executed successfully. The input image was loaded.

